

Slurm AI Agent

面向 USTC 107 算力平台的智能作业助手

作品简介

用自然语言降低使用门槛，用确定性后端守住执行边界

项目关键词

HPC 智能体

Slurm

Function Calling

依赖自愈

资源核验

知识检索

1 系统概述

Slurm AI Agent 是部署在 USTC 107 算力平台登录节点上的 Web 工作区：项目文件、Python 环境、作业提交、运行报告，以及接入大模型的智能助手，整合在同一个页面中。用户以自然语言描述任务目标，系统将其转化为真实的 Slurm 与 Conda 操作；每一步都在用户确认与后端校验之下完成。



图 1: 系统全貌：以项目为单元，组织从文件、环境到作业与复盘的全流程。

从准备依赖到拿到结果，系统围绕一条闭环展开：



集群与作业看板	展示节点、CPU/GPU 使用、实时作业、个人历史任务和提交选项。
项目工作空间	创建项目和实验子目录，上传、编辑、预览、下载文件，并维护独立会话。
智能助手	查询真实集群信息，检索平台文档，读取项目文件，解释配置并分析日志。
依赖管理	扫描依赖清单、源码 import 和说明文档，生成结构化建议，由用户确认后安装并验证。
作业规划与提交	提供 PyTorch 单卡/DDP、CPU、Job Array、Jupyter 和通用脚本模板；统一校验后提交。
监控与报告	持续跟踪作业，结束后生成报告，支持结果文件在线浏览与打包下载。
Web 终端与文档	在浏览器内使用登录 shell，并阅读或问答 107 平台文档。

2 为什么需要它

107 平台为课程训练、科学计算和深度学习提供了共享算力，但对许多用户而言，从写完代码到拿到结果之间仍有大量环节需要完成：理解分区、账户、QoS 与资源参数的含义，编写 sbatch

脚本，配置 Python 环境，再在作业排队或失败后查找日志、定位原因。一次任务往往需要在平台文档、终端、文件系统和作业队列之间频繁切换，单个环节并不复杂，串联起来却消耗大量时间与注意力。

使用中的高频障碍集中在五类，它们大多并非源于代码本身，而是作业生命周期中繁琐环节过多：

- 资源参数组合错误，提交即失败；
- 依赖缺失或版本冲突，反复出现；
- 作业排队或失败后，原因难以判断；
- 日志散布于各运行目录，难以定位；
- 报错为英文堆栈，初学者难以找到修改入口。

Slurm AI Agent 正是针对这些问题设计的。它不是简单地“让模型执行命令”，而是让模型负责它擅长的理解与解释，让程序负责它擅长的校验与执行。

3 典型使用流程

以下以一个典型任务为例，从上传代码到复盘结果依次经过五个环节。

3.1 第一步：创建项目与环境准备

用户通过 SSH 隧道打开 Web 控制台，创建项目并上传代码。每个项目对应平台上的一个目录：代码、数据放在项目根下，实验按运行子目录组织，所有提交脚本、标准输出、错误日志和报告统一归入 `logs/`。文件管理器支持树状浏览、在线编辑、预览和上传，路径校验拒绝越界访问项目目录之外的位置。系统在后台为项目创建专属 Conda 环境，就绪后自动通知；同项目的多个实验共享这套依赖，又各自保留独立的对话上下文。

3.2 第二步：检查依赖

点击“检查依赖”后，系统扫描项目里的依赖清单（`requirements.txt`、`environment.yml`、`pyproject.toml` 等）、Shell 脚本与 README 中的安装说明、以及 Python 源码中的 `import` 语句，生成一份依赖清单；随后向真实软件源逐项查询：该包是否存在、请求的版本是否可满足、环境是否已安装，结果直接标注在清单上。来自显式声明的可信依赖默认勾选，已安装的自动排除，拿不准的 `import` 候选交由模型复核并附上出处证据，供用户判断。

安装时，用户看到的不是抽象的进度条，而是 `conda/pip` 的真实终端输出。如果安装失败，系统先自动分类原因：求解冲突或版本不存在会自动去掉版本约束重试一次；磁盘、网络、包名错误等问题直接说明原因并终止；其余情况由模型结合真实版本查询结果给出修正方案，经用户确认后再继续。安装支持中断后增量重试，全部完成后系统逐包核验确实已安装至项目环境。

3.3 第三步：描述任务

用户用自然语言描述训练或计算目标，助手读取当前目录与集群实时状态，建议资源配置并生成命令正文。项目内的对话助手接入了 19 项受控工具：查询节点、分区、QoS、作业与历史用

量，检索平台文档，读取项目文件等。它回答“这个分区现在还剩多少 GPU”“我的作业为什么在排队”时，数据一律来自实时接口而非模型记忆。平台 Markdown 文档内置为本地知识库，支持语义问答；对平台概念不熟悉时，可直接向助手提问，无需在文档站中自行检索。

3.4 第四步：确认并提交

系统内置单卡 PyTorch、多机 DDP、CPU 批处理、Job Array、Jupyter 和通用脚本六类模板，参数默认值与范围都已预置。用户核对资源与命令后提交。提交前，后端完成一组用户容易遗漏的检查：作业名与路径是否合法、CPU 分区上是否误申请了 GPU、资源请求是否超出账户与 QoS 的授权上限、入口脚本与配置文件是否存在、运行目录是否重复。通过校验后，脚本头、运行目录、Conda 激活和日志路径由服务端锁定生成，用户的命令正文原样嵌入。

提交成功并不等于资源申请生效：系统会立即读取作业详情，用 TRES/GRES 证据核验 GPU 请求数，证据不足时如实标记为待确认。

3.5 第五步：监控与复盘

提交成功的作业进入持续观察列表，运行期间可随时查看状态、队列位置与实时日志。任务结束后，系统聚合 Slurm 状态与标准输出、错误日志的尾部，自动生成报告：成功作业给出结果概况、关键指标与警告；失败作业给出关键错误、原因分析与修复步骤。即使大模型暂不可用，报告也退化为带原始日志上下文的基础版本，保证信息不缺失。失败日志中明确的缺失依赖（如 `ModuleNotFoundError`）会被抽取出来写回项目依赖需求，下一次“检查依赖”时自动吸收这些运行反馈。

4 设计理念：人做决策，模型做理解，程序做约束

系统在整体上遵循一条清晰的职责划分。

- **人做决策。**安装哪些依赖、申请多少资源、何时提交、是否接受修正方案——所有有实际后果的决定都由用户确认，系统如实呈现决策所需的依据。
- **模型做理解。**大模型负责解释 Slurm 概念、阅读项目文件与平台文档、生成作业命令正文、诊断失败日志。它的输出是“结构化的建议”，而不是直接生效的操作。
- **程序做约束。**权限、配额、路径、脚本前导、环境激活与资源字段全部由后端锁定生成，模型无法绕过；提交之后系统还会回到 Slurm 读取真实结果进行核验。

另一条贯穿始终的原则是**证据优先**：队列状态、资源占用、配额这类随时变化的信息，必须通过工具实时获取，不允许模型凭记忆作答；回答保留数据来源，检索或分析失败时有确定性的降级方案，而不是沉默或编造。

5 创新点

以下几条均可对应到系统中的具体实现，而非泛化的工程实践。

1. **模型与执行权解耦的受控提交。**大模型只产出命令正文与结构化资源字段；脚本头、运行目录、Conda 激活与日志路径由后端锁定生成；提交后再次读取作业详情，用 TRES/GRES 证

据核验 GPU 请求是否生效，核验不可用时如实标记，避免“接口成功但资源未申请到”的误判。

2. **面向具体平台的模型校准**。系统提示词内置了在该集群实测确认的平台事实：分区名称区分大小写、分区与计费账户及 QoS 的对应关系、默认配额，以及 `scrontab` 被禁用（周期性任务建议改为长时限作业内循环）；同时约定数据口径——当前队列数量必须由实时工具逐条统计，控制器累计统计不得混作当前状态。模型回答因此贴合本平台实际，而非通用 HPC 印象。
3. **带证据链的知识检索**。平台 Markdown 文档按标题分块并向量化，向量缓存到磁盘；检索按余弦相似度排序，低于阈值的片段直接丢弃；回答保留来源文件与相似度，向量服务不可用时自动降级为关键词检索。
4. **依赖安装的预检与分类自愈**。安装前向真实软件源逐项查询：包是否存在、请求版本是否可满足、环境是否已安装，针对“把其它集群 `module` 的版本号当成 `conda` 版本”的高频错误按同系列优先给出建议版本；安装失败先由确定性分类器归类：常见冲突自动去版本重试，环境类问题直接终止并说明原因，仅少数疑难情形交给模型生成修正命令，且必须经用户确认；超时采用“停滞判定”而非固定总时长，避免把大体积包的健康下载误判为卡死。
5. **运行反馈驱动的环境闭环**。多实验共享项目依赖而会话相互隔离；作业结束后报告自动回到原会话，失败日志中确定性提取的缺失依赖写回项目需求，使下一轮“检查依赖”吸收真实运行反馈。

6 技术实现与安全边界

6.1 技术栈与规模

项目后端采用 FastAPI，前端为无需构建的原生单页应用；聊天与安装进度使用 SSE，终端使用 WebSocket + PTY（支持 ANSI 颜色与窗口缩放，交互式调试不必另开 SSH 客户端）。源码包含 49 个 HTTP/WebSocket 路由和 19 个智能体工具。

6.2 模型与调用方式

- 默认对话模型：`deepseek-v4-flash`；用户可在 Web 界面从当前 Key 可用的 `deepseek/glm` 模型中切换。
- 文档向量模型：`qwen3-embedding`，用于平台知识库语义检索；预留 `qwen3-reranker` 用于未来的检索重排序扩展。

系统通过 OpenAI Python SDK 调用学校提供的 OpenAI 兼容接口 (<https://api.llm.ustc.edu.cn/v1>)。对话使用 Chat Completions 与 Function Calling：请求传入多轮 `messages` 与 19 项工具定义，模型返回 `tool_calls` 时由后端执行真实工具并把结果回填给模型，直至生成最终回答。知识库使用 Embeddings 接口将 Markdown 片段与问题转换为向量，按余弦相似度检索；接口异常时自动降级为关键词检索。

6.3 与 Slurm 的交互与安全机制

Slurm 操作不经过大模型 API。系统通过 `slurmrestd v0.0.41` 调用作业、节点、QoS、历史和提交接口，使用 JWT 请求头认证；REST 未覆盖的优先级、fairshare 和实时用量信息，仅通过白名单只读 CLI 获取。在此基础上，系统实现了多项面向真实运行环境的工程机制：

- 密钥来自环境变量 `LLM_API_KEY` 与 `SLURM_JWT`（或由 `scontrol token` 自动刷新），均不写入仓库；系统默认通过 Unix Socket 与 SSH 隧道访问，不公开暴露登录节点 TCP 端口，终端连接做同源校验与并发限制；
- 项目级 Conda 环境采用线程锁与文件锁双层互斥，中断后的半成品环境可自愈重建；依赖安装采用停滞判定超时、失败分类器与信号阶梯终止，避免误杀健康下载或留下残缺环境；
- 路径穿越防护与白名单 CLI，模型与前端均无法越出项目目录或拼装任意命令；
- Slurm JWT 自动刷新、知识检索降级、作业报告兜底，外部服务异常时核心功能不中断。

7 适用对象与场景

- **课程与初学者**：第一次接触 Slurm 与 Linux 集群的学生，可以在引导下完成从上传代码到拿到训练结果的完整闭环，而不必先吃透全套命令。
- **科研训练与批量实验**：需要反复提交、对比和复盘的研究者，可以借助项目化空间、作业模板与自动报告减少重复操作。
- **教学与平台管理**：系统保留了清晰、可审计的权限与执行边界，适合在多用户共享登录节点上安全部署。

对新用户，它提供贯穿整个作业生命周期的引导；对有经验的用户，它减少查看资源、组织实验和复盘日志的重复操作；对平台管理场景，它保留了清晰的权限与执行边界。系统的目标始终一致：让用户把精力集中在计算任务本身，而不是消耗在工具切换和易错配置上。